R documentation

of all in 'man'

August 30, 2016

R topics documented:

baserateCT 1
baserateSet 2
codeSet 3
contingencyTable 3
createScS 4
kappaCT 5
kappaSet 5
rho 6
rhoCT 7
rhoMinTSLength 8
rhoR 9
rhoSet 10
testSet 11

Index **12**

`baserateCT` *Calculate Baserate (CT)*

Description

This function will calculate the baserate of the first rater, second rater, and the average of both the raters.

Usage

```
baserateCT(CT)
```

Arguments

`CT`	The `contingencyTable` for which the baserate is calculated

Details

A baserate is the percentage, as a decimal, that a positive code appears in a codeSet for a given rater. It is assumed that the first rater is more experienced and thus provides a better estimation of the actual baserate for a given code, so the first rater's baserate is often used as if it is the actual baserate. If the raters are assumed to have the same experience level, the average baserate may give a better estimation. If the second rater is more experienced, the second rater's baserate may give a better estimation. Functions assume that the first rater is the more experienced rater and thus uses the first rater's baserate as the overall baserate estimation.

Value

A list of the format:

firstBaserate The percentage that a positive code, or a 1, appears in the first rater

secondBaserate The percentage that a positive code, or a 1, appears in the second rater

averageBaserate The average percentage that a positive code, or a 1, appears in either of the two raters

See Also

baserateSet

---

baserateSet *Calculate Baserate (Set)*

Description

This function will calculate the baserate of the first rater, second rater, and the average of both the raters.

Usage

```
baserateSet(set)
```

Arguments

set	The codeSet for which the baserate is calculated

Details

A baserate is the percentage, as a decimal, that a positive code appears in a codeSet for a given rater. It is assumed that the first rater is more experienced and thus provides a better estimation of the actual baserate for a given code, so the first rater's baserate is often used as if it is the actual baserate. If the raters are assumed to have the same experience level, the average baserate may give a better estimation. If the second rater is more experienced, the second rater's baserate may give a better estimation. Functions assume that the first rater is the more experienced rater and thus uses the first rater's baserate as the overall baserate estimation.

Value

A list of the format:

firstBaserate The percentage that a positive code, or a 1, appears in the first rater

secondBaserate The percentage that a positive code, or a 1, appears in the second rater

averageBaserate The average percentage that a positive code, or a 1, appears in either of the two raters

See Also

baserateCT

---

codeSet *codeSet*

Description

A codeSet is a Nx2 binary matrix in which the first column corresponds to the first rater and the second column corresponds to the second rater.

Usage

```
codeSet
```

Format

The codeSet is an object of class matrix with n rows and two columns.

Examples

```
#An example codeSet
gold = c(1,1,1,1,rep(0,36))
silver = c(1,1,1,0,1,1,rep(0,34))
exampleSet = cbind(gold,silver)

#This set is included in the package under the variable name "codeSet".
```

---

contingencyTable *contingencyTable*

Description

A contingency Table is a 2x2 matrix that contains the counts of all combinations of positive and negative ratings made by two raters. If put in the form of a codeSet, the sum of all the combinations is equivilant to its length.

Usage

```
contingencyTable
```

Format

The contingency Table is an object of class matrix with two rows and two columns. The ordering of the combination vector input to the matrix is as follows: c(PP, NP, PN, NN), where the first letter represents the first rater and the second letter represents the second rater.

Examples

```
#A example contingencyTable
ct = matrix(c(3,2,1,34), nrow = 2, ncol = 2)

#This contingencyTable is included in the package under the variable name "contingencyTable".
```

`createScS`  *Create Simulated codeSet*

Description

Creates a simulated `codeSet` based off the given parameters

Usage

```
createScS(length, baserate, kappaMin, kappaMax, precisionMin, precisionMax)
```

Arguments

`length`	the length of the simulated `codeSet` to be created
`baserate`	the `baserateSet` of the simulated `codeSet`
`kappaMin`	the minimum kappa of the simulated `codeSet`
`kappaMax`	the maximum kappa of the simulated `codeSet`
`precisionMin`	the minimum precision of the simulated `codeSet`
`precisionMax`	the maximum precision of the simulated `codeSet`

Details

`codeSet`s are generated by first picking a random kappa within its range and a random precision within its range. If the random kappa, random precision, and baserate are not mathematically possible, then the precision is resampled from a range of mathematically possible values. A unique simulated `codeSet` is then constructed given these parameters.

Value

A simulated `codeSet` that fulfills the given parameters

kappaCT *Calculate kappa (contingency Table)*

Description

This function calculates Cohen's kappa given a contingencyTable

Usage

```
kappaCT(ct)
```

Arguments

ct	A contingencyTable

Value

The kappa of the contingencyTable

See Also

kappaSet

Examples

```
#Given a contingency Table
kappaCT(contingencyTable)
```

kappaSet *Calculate kappa (Set)*

Description

This function calculates Cohen's kappa for a given codeSet

Usage

```
kappaSet(set)
```

Arguments

set	A codeSet

Value

The kappa of the codeSet

See Also

kappaCT

Examples

```
# Given a set
kappaSet(codeSet)
```

---

rho *Rho*

Description

This function will calculate rho given the parameters of an observed testSet

Usage

```
rho(observedKappa, OcSBaserate, testSetLength, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

Arguments

observedKappa	The observed kappa value that will be tested with rho
OcSBaserate	The baserateSet of the observed codeSet
testSetLength	The length of the testSet the *observedKappa* was calculated from
testSetBaserateInflation	The minimum baserateSet gaurunteed for the testSet
OcSLength	The length of the observed codeSet
replicates	The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study
ScSKappaThreshold	The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSKappaMin	The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMin	The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMax	The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho

Value

rho for the given parameters

Examples

```
# Given only required parameters
rho(observedKappa = 0.88, OcSBaserate = 0.2, testSetLength = 80)

# Given required parameters plus testSetBaserateInflation and runs
rho(
 observedKappa = 0.88, OcSBaserate = 0.2, testSetLength = 80,
 testSetBaserateInflation = 0.2, replicates = 500
)
```

---

rhoCT *Rho (contingency Table)*

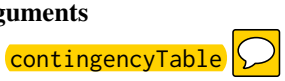
Description

This function calculates rho and kappa for a given contingencyTable, and returns a list containing both values.

Usage

```
rhoCT(contingencyTable, OcSBaserate = NULL, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

Arguments



contingencyTable	The contingencyTable used to calculate rho
OcSBaserate	The baserateSet of the observed codeSet
testSetBaserateInflation	The minimum baserateSet gaurunteed for the testSet
OcSLength	The length of the observed codeSet
replicates	The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study
ScSKappaThreshold	The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSKappaMin	The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMin	The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMax	The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho

Value

A list of the format:

rho The rho of the contingencyTable

kappa The Cohen's Kappa of the contingencyTable

Examples

```
# Given testSetBaserateInflation
rhoCT(contingencyTable, testSetBaserateInflation = 0.2)

# Given ScSBaserate and testSetBaserateInflation
rhoCT(contingencyTable, OcSBaserate = 0.1, testSetBaserateInflation = 0.2)
```

---

rhoMinTSLength *Rho Min testSetLength*

Description

This function calculates the minimum testSetLength where it is possible to get a rho less than alpha for the given parameters of rho.

Usage

```
rhoMinTSLength(baserate, alpha = 0.05, inc = 10, printInc = FALSE, ...)
```

Arguments

baserate	A baserateSet
alpha	The threshold of significance for rho (similar to an alpha level for a p value), defaulted to 0.05
inc	An integer indicating by how much the testSetLength should increase each iteration
printInc	A boolean indicating whether to print out each increment value with it's corresponding significance for rho
...	Any additional parameters passed into rho

Value

The minimum length of testSet, as a denomination of inc, that would give a value where rho becomes mathematically possible

Examples

```
#Add testSetBaserateInflation as an additional parameter
rhoMinTSLength(0.2, testSetBaserateInflation = 0.33)

#Add testSetBaserateInflation as well as changing inc and selecting printInc
rhoMinTSLength(0.2, inc = 5, printInc = TRUE, testSetBaserateInflation = 0.33)
```

rhoR *rhoR: A package for computing rho.*

Description

rho is used to test the generalization of inter rater reliability (IRR) statistics, in this case Cohen's Kappa.

Details

Rho starts with an observed kappa value, calculated on a subset of a codeSet, known as an observed testSet, and a *kappa threshold* which indicates what is considered significant agreement between raters.

It then generates a collection of fully-coded, simulated codeSets (ScS), further described in createScS, all of which have a kappa value below the kappa threshold and similar properties as the original codeSet.

Then, kappa is calculated on a testSet sampled from each of the ScSs in the collection to create a null hypothesis distribution. These testSets mirror the observed testSet in their size and sampling method. How these testSets are sampled is futher described in testSet



The null hypothesis is that the observed testSet, was sampled from a data set, which if both raters were to code in its entirety would result in a level of agreement below the kappa threshold.

Using an alpha level of 0.05, if the observed kappa is greater than 95 percent of the kappas in the null hypothesis distribution, the null hypothesis is rejected. Then one can conclude that the two raters would have acceptable agreement had they coded the entire data set. Of courese, researchers can set their own alpha level.

rho

Use rhoSet
rhoCT

kappa

Use kappaSet
kappaCT

rhoMinTSLength

Use rhoMinTSLength

rhoSet *Rho (set)*

Description

This function calculates rho and kappa for a given set, and returns a list containing both values.

Usage

```
rhoSet(set, OcSBaserate = NULL, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

Arguments

set	The test codeSet used to calculate rho
OcSBaserate	The baserateSet of the observed codeSet
testSetBaserateInflation	The minimum baserateSet gaurunteed for the testSet
OcSLength	The length of the observed codeSet
replicates	The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study
ScSKappaThreshold	The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSKappaMin	The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMin	The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho
ScSPrecisionMax	The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho

Value

A list of the format:

rho The rho of the codeSet

kappa The Cohen's Kappa of the codeSet

Examples

```
# Given testSetBaserateInflation
rhoSet(codeSet, testSetBaserateInflation = 0.2)

# Given ScSBaserate and testSetBaserateInflation
rhoSet(codeSet, OcSBaserate = 0.1, testSetBaserateInflation = 0.2)
```

`testSet` *Get Test Set*

Description

This function gets a *testSet* from a larger `codeSet` given certain sampling parameters.

Usage

```
testSet(set, testSetLength, testSetBaserateInflation)
```

Arguments

`set`	The `codeSet` from which the *testSet* is taken
`testSetLength`	The length of the *testSet* to be taken
`testSetBaserateInflation`	The minimum gaurunteed `baserateSet` of the *testSet*

Details



A *testSet* is a `codeSet` that is a subset of a larger `codeSet` with a given set of properties. A *testSet* is taken by first gaurnteeing a certain `baserateSet` as given by *testSetBaserateInflation*. The rest of the set is then randomly sampled from the original `codeSet`.

Value

A `codeSet` with the properties specified

Index

∗Topic **baserateCT**
baserateCT, 1
∗Topic **baserateSet**
baserateSet, 2
∗Topic **codeSet,**
createScS, 4
∗Topic **create**
createScS, 4
∗Topic **datasets**
codeSet, 3
contingencyTable, 3
∗Topic **kappaCT**
kappaCT, 5
∗Topic **kappaSet**
kappaSet, 5
∗Topic **rhoCT**
rhoCT, 7
∗Topic **rhoMinTSLength**
rhoMinTSLength, 8
∗Topic **rhoSet**
rhoSet, 10
∗Topic **rho**
rho, 6
∗Topic **testSet**
testSet, 11

baserateCT, 1, *3*
baserateSet, *2*, 2, *4*, *6–8*, *10, 11*

codeSet, *2, 3*, 3, *4–7*, *9–11*
contingencyTable, *1*, 3, *5*, *7, 8*
createScS, 4, *9*

kappaCT, *5*, 5, *9*
kappaSet, *5*, 5, *9*

rho, 6, *8*
rhoCT, 7, *9*
rhoMinTSLength, 8, *9*
rhoR, 9
rhoR-package *(*rhoR*)*, 9
rhoSet, *9*, 10

testSet, *6, 7*, *9, 10*, 11